B.Tech. Computer Science and Engineering

COURSE PLAN: THEORY COURSE

School Name	**School of Computer Engineering**				
Course Name	**Finite Automata and Compiler Design**				
Course Code	**CSE3128**	**Semester**	**V**	**Core / PE / OE**	**Core**
Name of the faculty	**Dr. Swathi Prabhu**				

No of contact hours/week:	**L**	**T**	**P**	**C**
	3	**1**	**0**	**4**

Mapping of Course Outcomes (COs)/Course Learning Outcomes (CLOs) #

CO/ CLO	Statements	No. of Contact Hours	Marks	Program Outcomes (POs)	Learning Outcomes #	BL* *(Recommended)*
1	Design grammars and automata for regular languages using the knowledge of grammars, languages, and automata.	16	30	1, 2, 7, 11	1, 2, 3	3 and above
2	Identify the steps involved in lexical analyzer generators by familiarizing the different phases of a compiler.	05	12	1, 2, 7, 11	2, 3	2 and above
3	Evaluate the top-down and bottom-up parsing techniques by comparing their effectiveness in various computational contexts.	16	33	1, 2, 3, 7, 8, 11	2, 3	3 and above
4	Analyze Syntax Directed Translation (SDT) techniques.	3	8	1, 2, 7, 11	3	3 and above
5	Analyze expressions to generate appropriate three address code.	6	12	1, 2, 7, 11	3	3 and above
6	Apply automated tools such as FLEX and YACC for lexical analysis and parser generation.	2	5	1, 5, 7, 8, 11	3, 12	3 and above
	Total	**48**	**100**			

Applicable to IET Accredited Programs Only

**Preferred BL levels are above 3 (apply)*

Course Articulation Matrix

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	2					1				1		2	
CO2	3	2					1				1	2	3	
CO3	2	3	2				1	1			1	2	3	
CO4	2	3	2				1				1		2	
CO5	2	2	1				1				1		2	
CO6	1				3		1	1			1	2	3	
Average Articulation Level	**2.1667**	**2.4**	**1.6667**	**0**	**3**	**0**	**1**	**1**	**0**	**0**	**1**	**2**	**2.5**	**0**

Module Matrix#

	LO1	LO2	LO3	LO4	LO5	LO6	LO7	LO8	LO9	LO10	LO11	LO12	LO13	LO14	LO15	LO16	LO17	LO18
CLO1	B	B	B															
CLO2		B	B															
CLO3		B	B															
CLO4			B															
CLO5			B															
CLO6			B									B						

Applicable to IET Accredited Programs Only. Fill with B or C

Delivery and Assessment Plan of LOs#

Learning Outcome (LO) mapped to the course		Delivery and assessment Plan
LO	**LO statement**	
LO1	Apply knowledge of mathematics, statistics, natural science and engineering principles to the solution of complex problems. Some of the knowledge will be at the forefront of the particular subject of study.	**Delivery:** Lectures, Tutorials. **Assessment:** Surprise tests (pen-and-paper), Semester examinations.
LO2	Analyse complex problems to reach substantiated conclusionsusing first principles of mathematics, statistics, natural science and engineering principles.	**Delivery:** Lectures, Tutorials. **Assessment:** MCQs, Semester examinations.

LO3	Select and apply appropriate computational and analytical techniques to model complex problems, recognising the limitations of the techniques employed.	**Delivery:** Lectures, Tutorials. **Assessment:** MCQs, Semester examinations.
LO12	Use practical laboratory and workshop skills to investigate complex problems.	**Delivery:** Lectures, Code Demonstrations. **Assessment:** Programming assignments, Implementation exercises, Semester examinations.

Applicable to IET Accredited Programs

The Assessment Plan shall be formulated as per the directives communicated by the Office of the AD(A) in every odd semester.

In semester & End semester plan and schedule (AY: 2026-27) Component	**Type**	**Max. Marks**	**Duration**	**Syllabus: Topics covered during**	**Schedule**	**Blooms taxonomy levels**
Assignment 1	Surprise test-Pen and Paper	10	20 min	L1 - L15	17 Aug 2026 – 29 Aug 2026	3 to 6
Assignment 2	Quiz	5	10 min	L16 - L24	7 Sep 2026 – 21 Sep 2026	2 to 6
Mid Sem Exam	Descriptive (E-pad)	30	90 min	L1 - L29	25 Sep 2026 to 1 Oct 2026	2 to 6
Assignment 3	Quiz	5	10 min	L30 - L42	19 Oct 2026 – 31 Oct 2026	2 to 6
END Semester examination	Descriptive (E-pad)	50	3 Hrs	L1 – L48	14 Nov 2026 – 28 Nov 2026	2 to 6

Lesson Plan

L No	Topics	CO Addressed
0	*To address OBE philosophy towards NBA and IET accreditation followed by a discussion on Course Plan, Course Outcomes, CO-PO Mapping and Blooms Taxonomy, with prerequisites for the course, if any.*	-
1	Three basic concepts: Languages and It's Operations	CO1
2	Grammar and Automata	CO1
3	Deterministic Finite Accepters, Languages and Dfa's, Regular Languages	CO1
4(T1)	Problems on Deterministic Finite Accepter	CO1
5	Nondeterministic Finite Accepters- Definition of a Nondeterministic Accepter, Why Nondeterminism	CO1
6	Equivalence of Deterministic and Nondeterministic Finite Accepters	CO1
7	Reduction of the Number of States in Finite Automata	CO1
8(T2)	Regular Expressions- Formal Definition of a Regular Expression, Languages Associated with Regular Expressions and examples	CO1
9	Connection between Regular Expressions and Regular Languages, Regular Expressions Denote Regular Languages, Regular Expressions for Regular Languages, Regular Expressions for Describing Simple Patterns,	CO1
10	Regular Grammars-Equivalence of Regular Languages and Regular Grammars	CO1
11	Identifying Non-regular Languages and Context-Free grammar	CO1
12(T3)	Examples on Context-Free grammar	CO1
13	Leftmost and Rightmost Derivations, Derivation Trees, Relation Between Sentential Forms and Derivation Trees, Parsing and Ambiguity- Ambiguity in Grammars and Languages	CO1
14	Nondeterministic Pushdown Automata- Definition of a Pushdown Automaton, The Language Accepted by a Pushdown Automaton	CO1
15	Examples on Nondeterministic Pushdown Automata	CO1
16(T4)	Examples Pushdown Automata and Context–Free Languages	CO1
17	Introduction to Language Processors, Structure of Compiler	CO2
18	The Role of the Lexical Analyzer	CO2
19	Input Buffering	CO2
20(T5)	Examples on Specifications and Recognition of Tokens	CO2
21	Problems on Transition-Diagram-Based Lexical Analyzer	CO2
22	Syntax Analysis – Introduction	CO3
23	Writing a Grammar- Lexical versus Syntactic Analysis	CO3
24(T6)	Problems on Eliminating Ambiguity and Left Recursion, Left Factoring	CO3
25	Top-Down Parsing - Recursive-Descent Parsers	CO3
26	First and Follow(1), LL(1) Grammars	CO3
27	Non-recursive Predictive Parsing	CO3
28(T7)	Examples on First and Follow(1), LL(1) Grammars, Non-recursive Predictive Parsing	CO3
29	Bottom-Up Parsing - Reductions, Handle Pruning	CO3
30	Shift-Reduce Parsing, Introduction to LR parsing- Simple LR, Why LR Parsers?	CO3
31	Items and LR (0) Automaton, The LR-Parsing Algorithm	CO3

32(T8)	Problems on Constructing SLR-Parsing Tables	CO3
33	Examples on LR parsing	CO3
34	More Powerful LR parsers- Canonical LR (1) Items	CO3
35	Constructing LR(1) Sets of Items, Canonical LR(1) Parsing Tables	CO3
36(T9)	Examples on Canonical LR(1) Parsing Tables	CO3
37	Conflicts During Shift-Reduce Parsing, Using Ambiguous Grammars-Precedence and Associativity to Resolve Conflicts	CO3
38	Syntax-Directed Translation - Syntax-Directed Definitions, Evaluation Order for SDD's- Dependency Graphs, Ordering the Evaluation of Attributes	CO4
39	Applications of Syntax-Directed Translation, Construction of Syntax Trees,	CO4
40(T10)	Examples on Syntax-Directed Definitions, Dependency Graphs	CO4
41	Intermediate-Code Generation - Variants of Syntax Trees, Directed Acyclic Graph for Expressions, The Value Number Method for Constructing DAGs, Three-Address Code, Addresses and Instructions, Quadruples, Triples, Static Single Assignment Form,	CO5
42	Problems on Three Address Code	CO5
43	Code Generation - Issues in Design of Code Generator,	CO5
44(T11)	Examples on Three Address code and Target Language Representation	CO5
45	Basic Blocks and Flow Graphs and Some Examples	CO5
46	The DAG Representation of Basic Blocks, Finding Local Common Subexpressions, Dead Code Elimination, Peephole Optimization and Peephole Optimization	CO5
47	Theory of FLEX- Structure of a FLEX program, Regular Expression, FLEX library functions	CO6
48(T12)	Theory of YACC- YACC Symbols, Symbol values, Symbol Types, YACC Library, Some Example Program	CO6

Self Directed Learning (SDL)

Sl. No.	TOPIC	No. of learning hours	CO
1	Formal Languages, Finite Automata and Pushdown Automata using NPTEL	6	CO1
2	Automata Design and Language Conversion using JFLAP	6	CO1
3	Building a Lexical Analyzer using Flex	6	CO2
4	Advanced Tokenization and Unicode Support in Modern Compilers	6	CO2
5	Parser Generation using ANTLR	6	CO3
6	Modern Syntax Error Detection and Recovery in Parser Generators	6	CO3
7	Attribute Grammar Systems and Incremental Semantic Analysis	6	CO4
8	Building a Domain-Specific Language (DSL) using ANTLR with Semantic Actions	6	CO4
9	LLVM Intermediate Representation (LLVM IR) and Modern Compiler Optimization	6	CO5
10	Machine-Independent Compiler Optimizations using Static Analysis	6	CO5
11	Case Study – Building a Calculator using Flex and GNU Bison	6	CO6

12	Research Survey – The Deep Learning Compiler: A Comprehensive Survey	6	CO6
		72	

***The SDL topics may be from the Research Literature/ Case Studies NPTEL/Coursera/any MOOC-based material Virtual Lab, etc.**

****One credit is equivalent to 30 hours of learning. (4 credits course =120 hrs learning ; 48+72=120 hrs of learning)**

Faculty members teaching the course (if multiple sections exist):

Faculty Name	Section	Signature	Faculty Name	Section	Signature
Dr.Gangothri Sanil	A		Dr Athira Gopalakrishnan	D	
Dr. Deepa G,	B				
Dr. Swathi Prabhu	C				

References:

Textbooks	1. Peter Linz, *an Introduction to Formal Languages and Automata, (6e),* Jones & Bartlett Learning, 2019.
	2. Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, *Compilers Principles, Techniques and Tools,* (2e), Pearson Education, 2013.
	3. John R. Levine, Tony Manson, Doug Brown, *LEX & YACC*, O Reilly Media, 2012.
	4. J E Hopcroft, Rajeev Motwani & Jeffrey D Ullman, *Introduction to Automata Theory, Languages and Computation*, (3e), Pearson Education, 2006.
	5. Kenneth C. Louden, *Compiler Construction - Principles and Practice, (1e)*, Thomson, 2007.
	6. Vinu V. Das, Compiler Design using FLEX and YACC, Prentice-Hall, 2007.
Self-Directed Learning	
Research Literature/ Case Studies	• LEX – A Lexical Analyzer Generator – https://thiagoh.github.io/lex/ • Unicode Standard Annex #31 – https://unicode.org/reports/tr31/ • LL(*): The Foundation of the ANTLR Parser Generator – https://www.antlr.org/papers/LL-star-PLDI11.pdf • Automatic Syntax Error Reporting and Recovery in Parsing Expression Grammars – https://doi.org/10.1016/j.scico.2019.102373 • LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation – https://llvm.org/pubs/2004-01-30-CGO-LLVM.pdf • GNU Bison Manual – https://www.gnu.org/software/bison/manual/ • GNU Bison Calculator Examples – https://www.gnu.org/software/bison/manual/html_node/Examples.html • The Deep Learning Compiler: A Comprehensive Survey – https://doi.org/10.1109/TPDS.2020.3030548
NPTEL/Coursera/any MOOC-based material	• Formal Languages and Automata Theory – https://nptel.ac.in/courses/111103016 • Compiler Design – https://nptel.ac.in/courses/106108052 • Compiler Design – https://nptel.ac.in/courses/106106131 • Compiler Design https://onlinecourses.nptel.ac.in/noc21_cs07/preview

Virtual Labs	• JFLAP – https://www.jflap.org/ • OnlineGDB – https://www.onlinegdb.com/ • ANTLR Playground – https://lab.antlr.org/ • Compiler Explorer (Godbolt) – https://godbolt.org/

Submitted by: Dr. Swathi Prabhu

Swathi

(Signature of the faculty)

Date: 7/8/2026

Approved by:

28/8/26

(Signature of the Dean)